

Programmazione Avanzata e parallela

Lezione 02

Cosa vedremo oggi

Organizzare progetti in C

- Compilazione separata (in C)
 - File header, object files
 - Librerie
- Opzioni del compilatore
 - Ottimizzazione
 - Debug

Cosa vedremo oggi

Make e Makefile

- Gestire il build di progetti con make
 - Cosa sono i makefile
 - Regole dei makefile
 - Utilizzo di variabili
 - Regole template
 - A cosa prestare attenzione

Compilazione semplice

Quello fatto fino a ora

```
int f(...)  
{  
    //...  
}  
  
int main(...)  
{  
    //...  
    int x = f(...)  
    //...  
}
```

main.c

gcc -o main main.c

main

Compilazione semplice

Quello fatto fino a ora

- E se volessimo usare una funzione definita in main.c in un'altro programma?
- Dovremmo copiarla
- Solitamente non facciamo copia e incolla di “printf” o “malloc” da altri file, quindi qualcosa di meglio deve essere possibile
- Idea: dividere le funzioni e le definizioni di strutture su più file
- Serve un modo per dire “questa funzione esiste da qualche altra parte”

Spezzare in più file

E object file

- Possiamo spezzare in più file
- Dobbiamo dire al compilatore di fermarsi in un momento “intermedio” in cui:
 - Il codice macchina è stato generato...
 - ...ma alcune funzioni non esistono ancora.
Pensate a un “branch a [TODO]”
- Servirà poi un passo finale per mettere assieme tutto (“linking”)

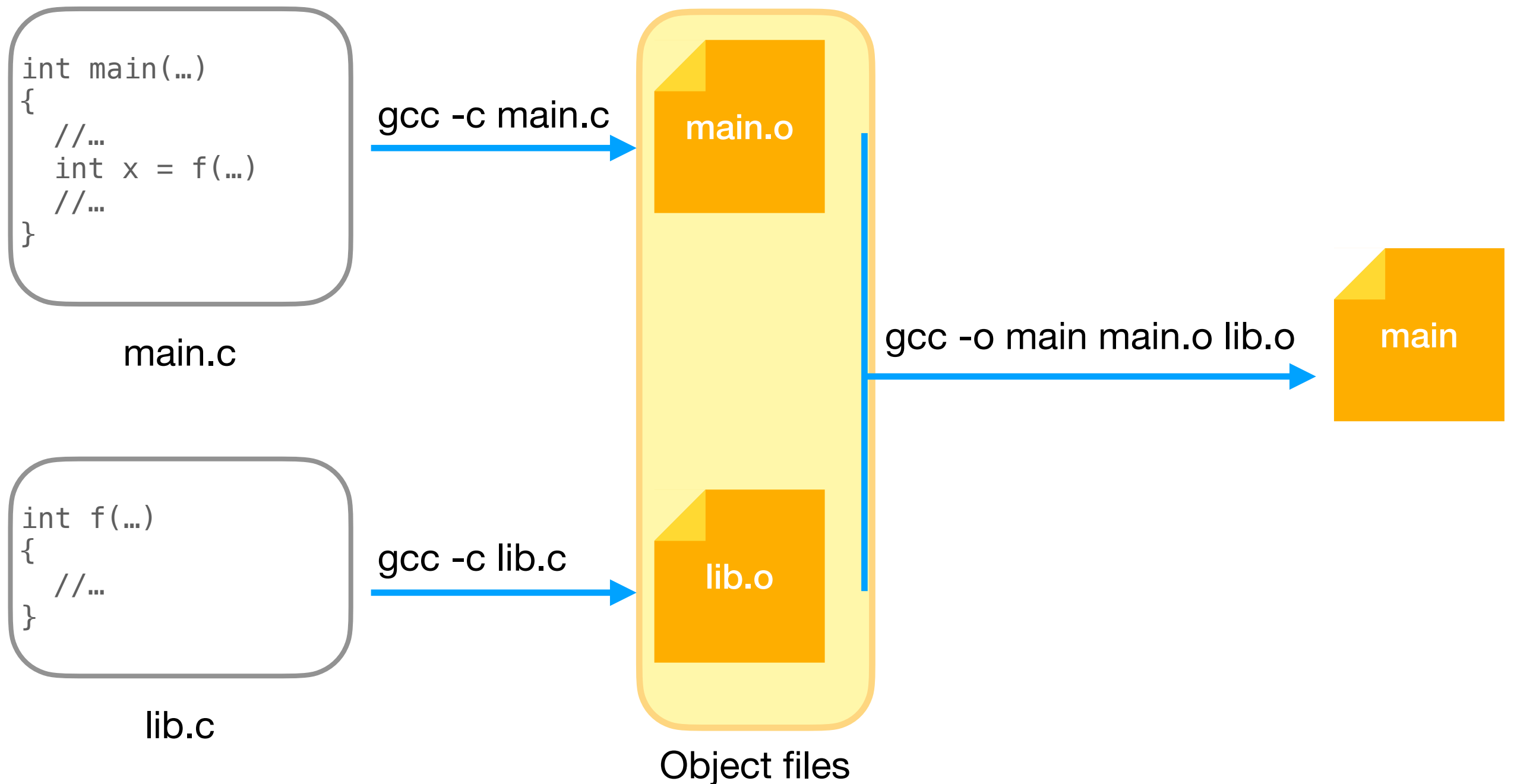
Spezzare in più file

Usare “extern”

- Dobbiamo poter dire che una funzione non definita nello stesso file “esiste”
- Per questo dichiariamo una funzione come “extern” e non la definiamo
- E.g., `extern int f(int x, int y);` in un file mentre un altro file conterrà l’effettiva definizione di `f`.
- Questo non può venir fatto con le strutture, però possiamo duplicare la definizione su più file (se è sempre la stessa non ci crea problemi)

Spezzare in più file

Processo di compilazione



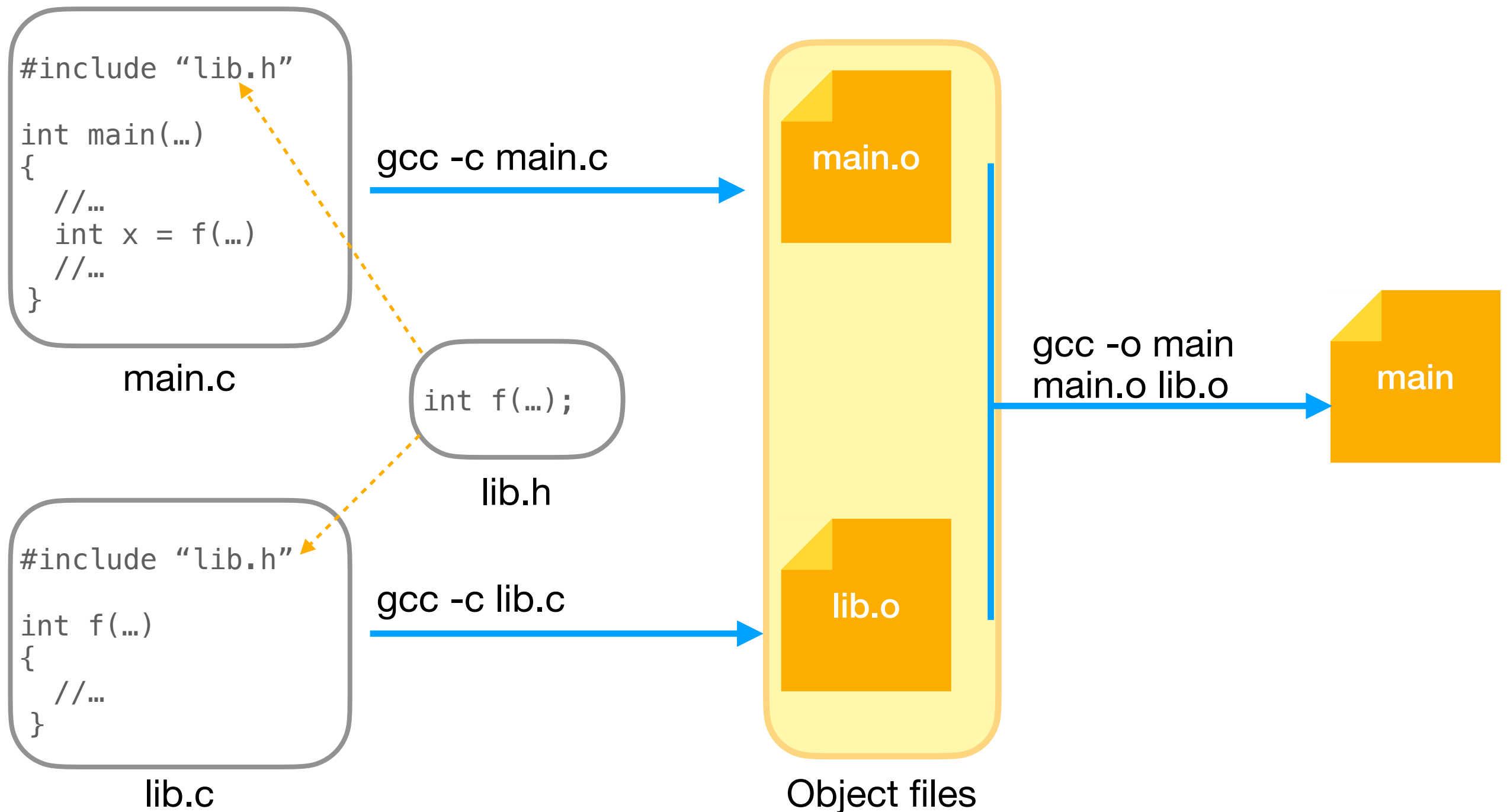
File Header

Gestire meglio la compilazione separata

- Per le funzioni l'uso di “extern” è scomodo
- E definire più volte le strutture può portare a problemi inattesi
- Sarebbe utile avere un file che dica “queste sono le strutture e le funzioni utilizzabili”
- Questo è il compito dei file header (solitamente con estensione .h)
- Questi sono inclusi con `#include “file.h”` o `#include <file.h>`
- La differenza è dove sono cercati i file header (directory locale o path definiti dal sistema)

Spezzare in più file

Processo di compilazione



File Header

Gestire meglio la compilazione separata

- Generalmente il file header include:
 - La dichiarazione delle funzioni
 - La definizione delle strutture da utilizzare
- È buona norma includere il file header anche nel file di implementazione (il corrispondente .c):
 - Per usare le definizioni di strutture
 - Per fare in modo che le dichiarazioni delle funzioni siano consistenti con la loro definizione

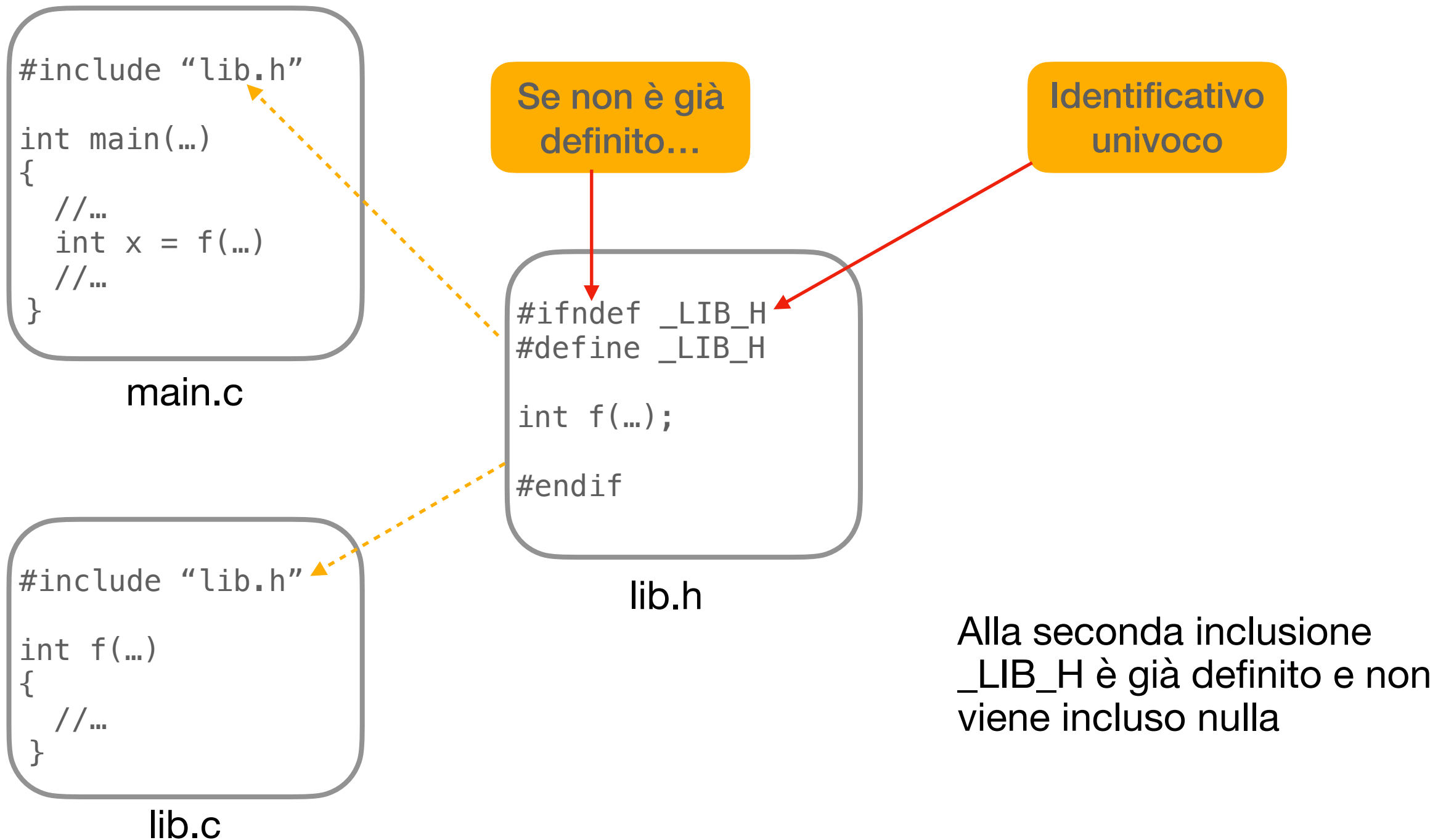
File Header

Inclusione multipla

- E se includiamo un file header più di una volta?
 - Per le dichiarazioni di funzioni nessun problema
 - Per le definizioni di strutture (e funzioni) invece c'è un problema di duplicazione della definizione!
- Verificare a mano di fare una sola inclusione è difficile
- Solitamente si risolve con delle macro del preprocessore C

File header

Evitare la doppia inclusione



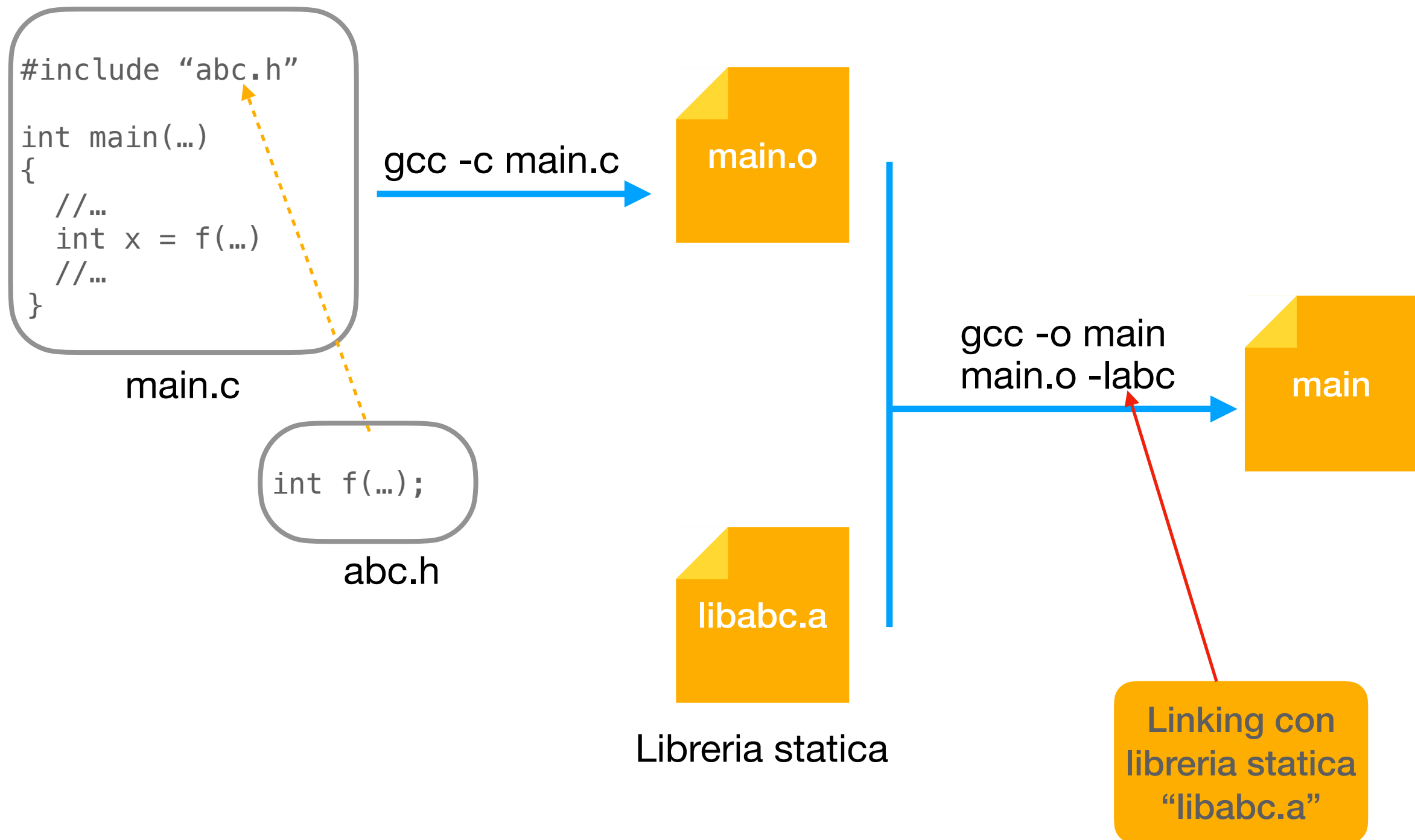
Librerie statiche

Rendere disponibile il vostro codice

- Una possibilità per rendere il codice (anche definito da più file) disponibile ad altri è tramite delle librerie
- Le librerie statiche sono incluse nel programma al momento del linking
- Le librerie dinamiche sono caricate quando il programma viene eseguito (ma devono essere presenti nel sistema)
- La creazione di librerie statiche è fatta tramite il comando “ar”
- È possibile esplorare il contenuto delle librerie statiche col comando “nm”

Librerie statiche

Processo di compilazione



Librerie statiche

Sequenza di creazione

- Innanzitutto compiliamo creando gli object file
`gcc -c file1.c file2.c`
- Mettiamo assieme i file con “ar”:
`ar r libabc.a file1.o file2.o`
- Ora serve creare un “indice” dei simboli della libreria (in pratica cosa la libreria fornisce):
`ranlib libabc.a`
- Alternativamente è possibile fare i due passi in una volta sola:
`ar rs libabc.a file1.o file2.o`

Librerie statiche

Utilizzo

- Per usare una libreria statica dobbiamo dire al compilatore due cose:
- Dove trovare la libreria. Questo è fatto tramite l'opzione `-L` a cui si aggiunge il path dove si trovano i file `.a`
- Dire al compilatore di fare linking con la libreria. Per questo si usa l'opzione `-l` con il nome della libreria *senza il “lib”* iniziale
- Quindi per linking con `libabc.a` nella directory attuale:
`gcc -L. -o main main.c -labc`

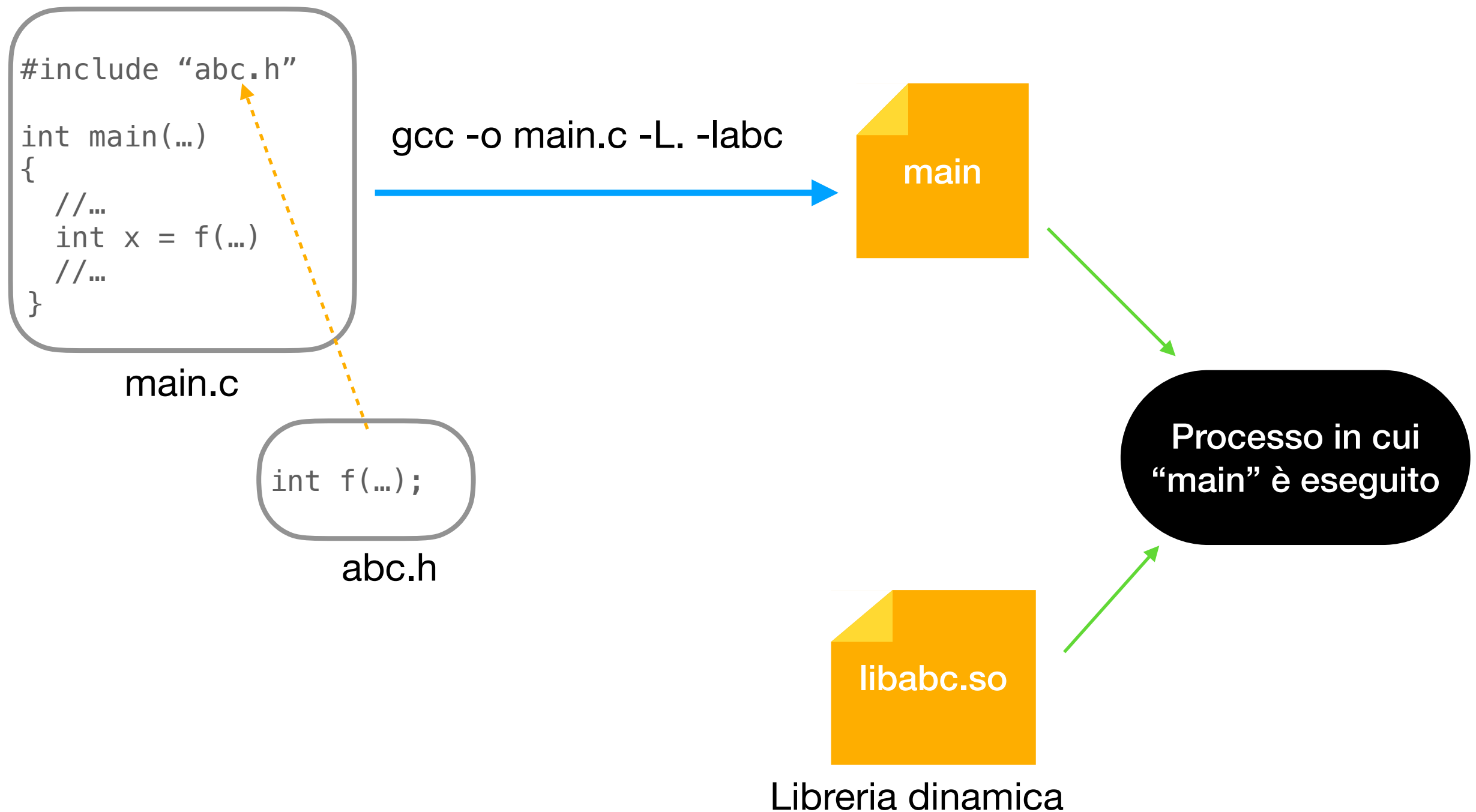
Librerie dinamiche

Rendere disponibile il vostro codice (2)

- Le librerie dinamiche sono caricate quando il programma esegue (sono i file .so in Linux, .dylib su macOS e .dll su Windows)
- Hanno come vantaggi che:
 - Potete aggiornarle senza ricompilare tutto
 - Più programmi possono usare la stessa libreria senza duplicare il codice nell'eseguibile (risparmiando spazio)
 - Potenzialmente potete caricare alcune librerie solo quando necessario
- La loro creazione è però un poco più complicata

Librerie dinamiche

Processo di compilazione



Librerie dinamiche

Difficoltà di creazione e uso

- Il codice generato deve essere “position independent”, dato che non è possibile stabilire al momento della compilazione in che indirizzi di memoria sarà messo
 - Questo viene fatto passando l'opzione -fPIC al compilatore
- Al momento dell'esecuzione il loader deve sapere dove trovare la libreria dinamica da caricare
 - In Linux (e diversi UNIX) si utilizza la variabile d'ambiente LD_LIBRARY_PATH
 - In macOS si utilizza la variabile d'ambiente DYLD_LIBRARY_PATH

Librerie dinamiche

Creazione

- Generiamo codice “position independent”:
`gcc -fPIC -c abc.c`
- Creiamo la libreria dinamica:
`gcc -o libabc.so -shared abc.o`
- Possiamo ora compilare usando la libreria dinamica:
`gcc -o main main.c -L. -labc`
- Possiamo vedere che il programma “main” compilato necessita di alcune librerie statiche usando i comandi:
`ldd main (Linux)`
`otool -L main (macOS)`

Librerie dinamiche

Utilizzo

- La posizione nel filesystem di una libreria dinamica può variare
- Alcuni path possono essere salvati nell'eseguibile
- Altri sono dati dalle variabili d'ambiente
`LD_LIBRARY_PATH` (Linux) o `DYLD_LIBRARY_PATH` (macOS)
- Possiamo vedere cosa contengono con
`echo $LD_LIBRARY_PATH`
- Possiamo aggiungere dei path (come “./lib”) alle variabili con:
`export LD_LIBRARY_PATH=./lib:$LD_LIBRARY_PATH`
- Cosa succede se la libreria non è nel path?

Opzioni comuni del compilatore

Ottimizzazione

- **-On** con $n \in \{0,1,2,3\}$:
 - **-O0** disattiva le ottimizzazioni
 - **-O3** è (solitamente) il massimo livello di ottimizzazione
- **-march=native** per dire di utilizzare tutte le estensioni (e ottimizzazioni) che hanno senso sulla macchina specifica (e non su una “generica” della stessa architettura)
- **-march=[nome]** di solito indica l’architettura target della compilazione
- Nota: su macchine più vecchie/diverse l’eseguibile potrebbe non funzionare!

Opzioni comuni del compilatore

Debugging

- **-g** compila coi simboli di debug
 - Rende più semplice utilizzare i debugger (e.g., gdb) mantenendo cose come i nomi delle variabili, etc.
 - Generalmente non consigliato con livelli di ottimizzazione elevati
- **-profile-generate** compila per permettere il profiling
 - Quando eseguito il programma genera un file con le informazioni su come il codice viene utilizzato
 - Che puoi aiutare le ottimizzazioni tramite **-fprofile-use**

Opzioni extra

Ancora qualche opzione del compilatore

- **-Wall**
abilita tutti i warning ragionevoli (non tutti!)
- **-pedantic**
abilita tutti i warning relativi al non rispetto dello standard (e.g., utilizzo di estensioni specifiche del compilatore)
- **-Wextra**
Ancora più warning di -Wall (non necessariamente da considerare tutti)
- **-std=c89** (o anche c90)
-std=c99
-std=c11
-std=c17 (o anche c18)
indica che versione del C si sta utilizzando (lo standard è stato aggiornato più volte negli anni)

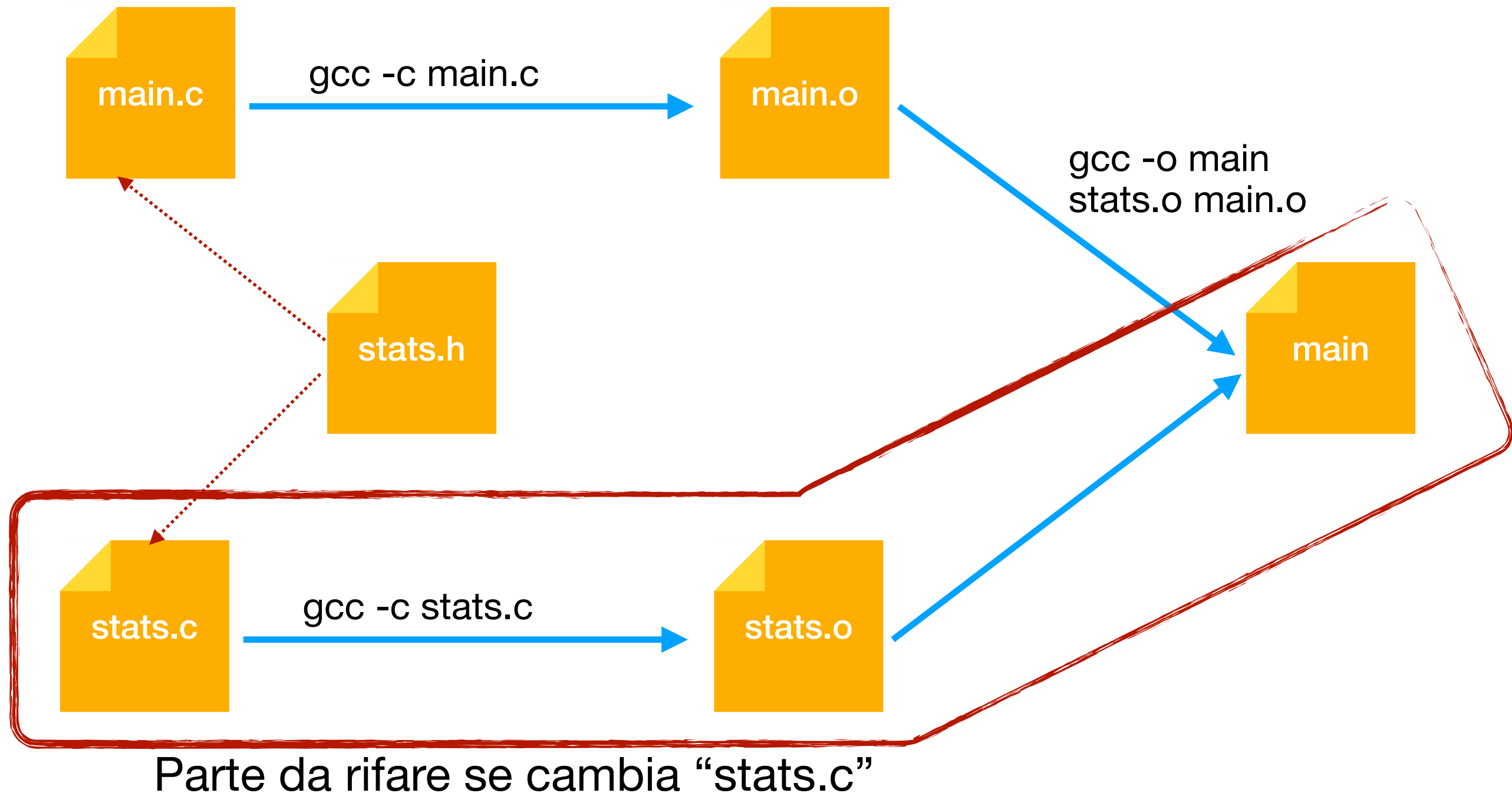
Make

Motivazioni per l'esistenza

- Se abbiamo decine o centinaia (o molte migliaia) di file .c e .h sarebbe utile gestire bene la compilazione separata
- Se modifico il file “test.c” devo sicuramente ricreare il file “test.o” e fare il linking
- Mi serve ricompilare tutto il resto?
- Se modifico il file “test.h”?
- Sarebbe utile avere un sistema che in automatico mi ricompila solo quello che serve

Passi di compilazione

E cosa servirebbe rifare



Make

Motivazioni per l'esistenza

- In generale vorremmo un sistema in cui possiamo specificare tre cose:
 - Che output vogliamo produrre (il file .o, l'eseguibile, etc)
 - Quali sono i prerequisiti per (e.g., avere i file .c e .h corretti, avere tutti i file .o)
 - Che comando eseguire per generare l'output dati i prerequisiti
- Il sistema dovrebbe poi essere in grado di trovare cosa serve per generare i prerequisiti se non li abbiamo (ma abbiamo una regola per generarli)

Make

Alcuni esempi

- Cosa vorremmo specificare:
 - *Obiettivo*: main
 - *Prerequisiti*: main.o, stats.o
 - *Comando*: gcc -o main main.o stats.o
 - *Obiettivo*: stats.o
 - *Prerequisiti*: stats.c, stats.h
 - *Comando*: gcc -c stats.c

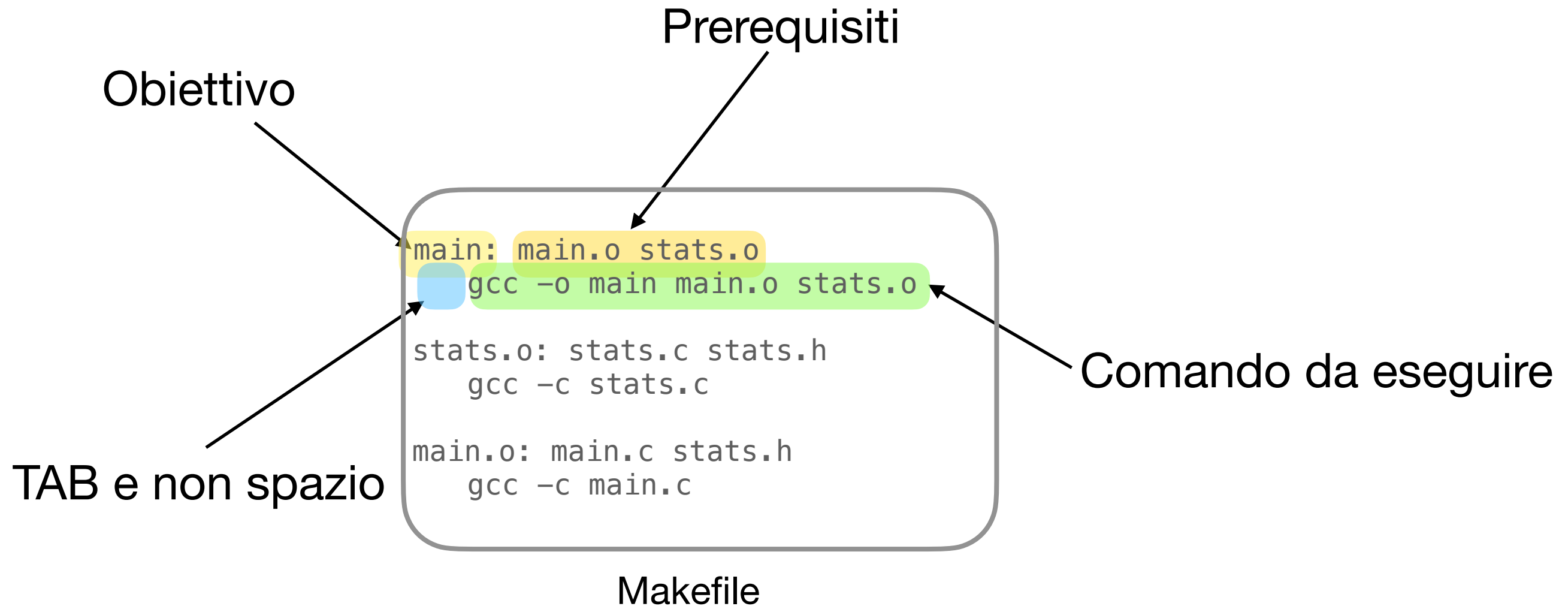
Makefile

Struttura di base

- Make è uno strumento che ci permette di specificare queste tre cose (obiettivi, prerequisiti e comandi da eseguire)
- La prima versione è del 1976, esistono molte implementazioni che, per le funzionalità essenziali, sono tutte compatibili
- Di default basta creare un file chiamato “Makefile” e invocando make quello sarà il file di comandi considerato
- L’idea è che invocando “make” posso fare tutti (e soli) i passi necessari a costruire la versione aggiornata del mio programma

Makefile

Esempio



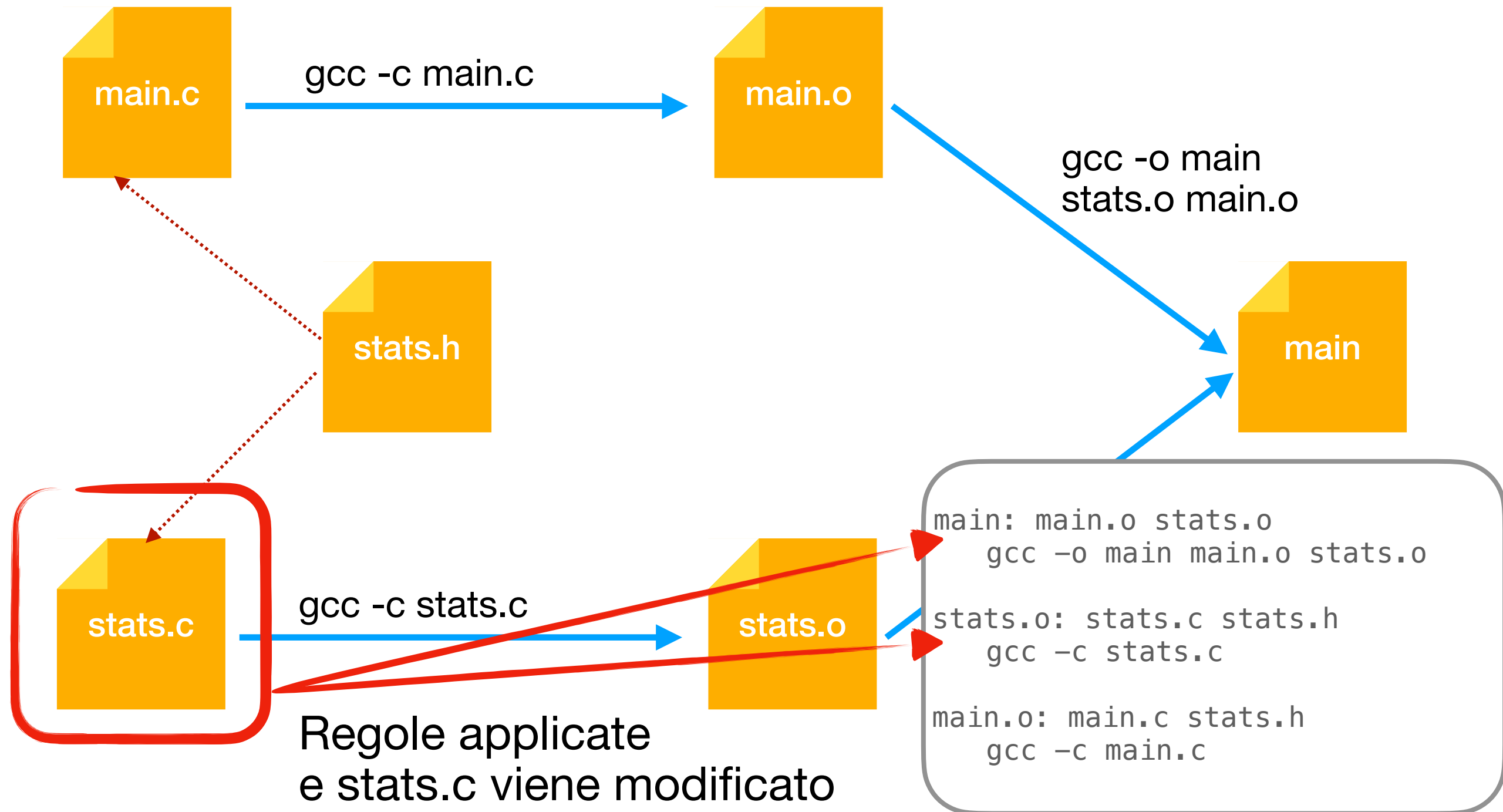
Regole del makefile

Quando sono invocate?

- Se i prerequisiti sono stati modificati...
- ...allora l'obiettivo deve essere ricostruito
- Per fare questo sono eseguiti i comandi indicati nella regola (posso essere anche vuoti)
- La verifica dei prerequisiti richiede una applicazione ricorsiva:
 - Se il prerequisito non esiste, cerca una regola per crearlo
 - Se il prerequisito esiste, verifica se deve essere ricostruito (applicando la procedura indicata qui)

Passi di compilazione

E cosa servirebbe rifare



Make

Invocazione

- Adesso che abbiamo un makefile possiamo invocare make:
 - `make main`
chiediamo di costruire il target “main”
 - `make -f nome_makefile main`
se vogliamo un nome specifico per il makefile
 - `make`
senza una indicazione del target viene richiamato il target “all”

Make

Invocazione

- Cosa succede se richiamiamo “make” una seconda volta?
 - “make: Nothing to be done for `all’.”
 - Se non modifichiamo nulla non ha senso rifare tutta la compilazione e il linking.
- E se modifichiamo un solo file .c?
 - Non vengono eseguiti tutti i comandi ma solo quelli in cui i prerequisiti sono stati modificati (quindi ricreazione del file .o e linking)

Make clean

Pulizia

- Un target comune è “clean”
- Non corrisponde a nessun file, ma semplicemente esegue una serie di comandi di pulizia dei file intermedi
- Esempio per eliminare i file oggetto:
 - clean:
rm *.o
- Dato che il file “clean” non viene creato il comando viene sempre eseguito
- Per evitare malfunzionamenti in caso esista un file “clean” usiamo la direttiva “.PHONY”, per dire di non considerare mai aggiornato il target

Makefile

Versione attuale

```
all: main

main: main.o stats.o
    gcc -o main main.o stats.o

stats.o: stats.c stats.h
    gcc -c stats.c

main.o: main.c stats.h
    gcc -c main.c

clean:
    rm *.o
```

Makefile

Possiamo fare dei target per generare librerie statiche e dinamiche?

E se volessimo mettere le librerie in una directory specifica?

E se volessimo cambiare le opzioni del compilatore?

Variabili

Nei makefile

- A volte vogliamo poter parametrizzare i comandi in un makefile e usare una variabile comune per tutti comandi
- Esempi:
 - Il nome del compilatore (gcc, gcc-13, clang, etc)
 - I flag di compilazione (-O3, -g, etc)
 - La directory in cui spostare i file (e.g., “lib”, “.”, etc)
- Per fare questo abbiamo a disposizione le variabili nei makefile

Variabili

Nei makefile

- Una variabile è definita come
NOME = stringa
- Da quel momento in poi possiamo usare **\${NOME}** nel makefile e verrà sempre sostituito da “stringa”
- Notate che è sempre una sostituzione del testo, non vi sono operazioni che vengono compiute
- Esempio (dichiarazione):
CFLAGS = -O3
- Esempio (uso):
gcc \${CFLAGS} -o main main.o ...

Variabili

Modifica da linea di comando

- Un vantaggio delle variabili è che possono essere modificate direttamente dalla linea di comando di make
- Esempio
make CC=gcc-13
utilizzerà un compilatore differente
- Questo permette di avere un default facilmente modificabile senza toccare il makefile
- Utile per le opzioni di compilazione (e.g., ottimizzazione, debug, profiling, etc).

Variabili automatiche

Nei makefile

- Alcune variabili esistono sempre e sono definite in automatico:
 - `$@`
il target
 - `^`
la lista dei prerequisiti
 - `<`
Il primo prerequisito
- Questo ci permette di evitare di riscrivere alcuni dei nomi già presenti in target e prerequisiti

Regole template

One rule to rule them all

- In molti casi ci sono regole simili che cambiano solo per il nome del file considerato (e.g., generazione del file .o dal file .c)
- Per fare questo ci vengono incontro delle regole template in cui possiamo usare “%” per indicare “qualsiasi sequenza di caratteri” nel target e poi usare “%” per indicare la stessa sequenza nei prerequisiti
- Esempio:
`%o: %.c`
`gcc -c $<`
 - Significato: per generare un file “nome.o” ti serve “nome.c”

Regole template

One rule to rule them all

- Quando usiamo le regole template abbiamo una variabile automatica aggiuntiva:
 - **\$***
Fa matching della parte template (ovvero %) per poterla usare nei comandi da eseguire
- Questa ha senso solo quando abbiamo una regola template

Wildcards

Tutto ciò che fa matching di un pattern

- A volte ci serve una variabile come “tutti i file.c” o “tutti i file.h”
- Per questo possiamo usare le wildcard
- `${wildcard *.c}`
produrrà una lista di tutti i file .c presenti nella directory corrente
- Alcune versioni di make, come GNU make, hanno la possibilità di fare maggiori manipolazioni aggiungendo prefissi, suffissi, o sostituendo sottostringhe:
https://www.gnu.org/software/make/manual/html_node/Functions.html

Esecuzione condizionale

Regole condizionali

- A volte è utile cambiare cosa è definito nel makefile in base a certe condizioni (e.g., l'ambiente di sviluppo usato)
- Abbiamo due possibilità:
 - Testare se due stringhe sono identiche
 - Testare se una variabile è definita
- La sintassi è lievemente differente

Esecuzione condizionale

Regole condizionali

ifeq “stringa1” “stringa2”

per verificare se una variabile è definita si utilizza ifdef

caso di uguaglianza

else

caso di non uguaglianza

endif

A cosa fare attenzione

Make è comunque degli anni '70

- Ogni riga dei comandi da eseguire all'interno di una regola viene eseguita in modo indipendente (i.e., non possiamo spezzare su più righe)
- È essenziale che ci siano dei tab per indentare, niente spazi.
- Utilizzando l'opzione “-p” è possibile stampare che comandi verranno eseguiti e che variabili e regole sono definite
- Aggiungendo @ prima di un comando non viene stampato il comando che viene eseguito

Altri sistemi di build

Cmake, bazel, etc

- Negli anni sono stati definiti molti sistemi di “build automation”, con più o meno successo. Alcuni che potete incontrare sono:
 - **CMake**. Non fa direttamente il build ma costruisce gli script/file (anche make) per fare il build con altri tool
 - **Autotools** (automake e autoconf). Tool per generare i makefile in maniera portabile
 - **Bazel**. Scritto in Java per effettuare build di progetti Java, C, C++, etc. Deriva da Blaze, il sistema di build interno di Google
 - **Gradle**. Della Apache foundation, più comune su progetti in Java (in quel caso trovate anche Ant e Maven)